Vectors and iterators

plus `unique_ptr`

Last time: the heap is easy to get wrong

Forget `delete`
Memory leak.

Use it after `delete`
Dangling pointer.

`delete` it twice
Undefined behavior.

Today: tools that handle the heap for you.

`vector`
A whole array that grows, knows its size, and cleans up after itself.

`iterator`
A position inside a container. Acts like a pointer.

`unique_ptr`
One heap object, one owner, deleted for you.

Vectors

arrays that manage themselves

What's wrong with plain arrays?

Fixed size

`int scores[5];` can never hold a sixth score.

They don't know their size

You have to keep track of it yourself.

No bounds checking

`scores[5]` reads past the end without complaint.

A `vector` fixes all three. It's part of the C++ standard library, in the section people usually call the **STL** (Standard Template Library).

To use one: `#include <vector>`

Declaring a vector

The type of thing the vector holds goes in angle brackets: `vector<int>`, `vector<string>`, `vector<double>`.

Line 8: an empty vector. It can grow, so it doesn't need a size.

Line 11: a vector that starts with 4 elements (4 empty strings).

Indexing works just like an array.

```
$ ./firstVector
Charli XCX
size: 4
```

```
6   int main() {
7       // an empty vector: no size needed, it can grow
8       vector<int> scores;
9
10      // or give it a starting size: 4 empty strings
11      vector<string> artists(4);
12
13      artists[0] = "Charli XCX";
14      artists[1] = "100 gecs";
15      artists[2] = "A. G. Cook";
16      artists[3] = "Danny L Harle";
17
18      cout << artists[0] << endl;
19      cout << "size: " << artists.size() << endl;
20
21      return 0;
22  }
```

Predict: reading past the end

`artists` has 4 elements, at indexes 0 to 3.

The loop runs while `i <= 4`. What happens on the fifth pass?

```
6   int main() {
7       vector<string> artists(4);
8       artists[0] = "Charli XCX";
9       artists[1] = "100 gecs";
10      artists[2] = "A. G. Cook";
11      artists[3] = "Danny L Harle";
12
13      for (int i = 0; i <= 4; i++) {
14          cout << artists[i] << endl;
15      }
16
17      return 0;
18  }
```

Answer: undefined behavior, again

`artists[4]` doesn't exist, and `[]` doesn't check. Same program, two machines:

g++ on Linux

```
$ ./pastEnd
Charli XCX
100 gecs
A. G. Cook
Danny L Harle
Segmentation fault
```

On a Mac, the original version of this program printed an extra **blank line** and finished normally.

On Linux, it crashes every time.

Neither one is “what it does.” There is no right answer: that's what undefined behavior means.

Range-`for`: the easy way to loop

Read line 12 as “for each `name` in `artists`.”

No index, no `size()`, no way to run off the end.

Line 7 shows another way to set up a vector: list its starting values in braces.

```
6   int main() {
7       vector<string> artists = {"Charli XCX",
8                                 "100 gecs",
9                                 "A. G. Cook",
10                                "Danny L Harle"};
11
12      for (string name : artists) {
13          cout << name << endl;
14      }
15
16      return 0;
17  }
```

```
$ ./rangeFor
Charli XCX
100 gecs
A. G. Cook
Danny L Harle
```

When you need the index: `size_t`

`size()` gives back a `size_t`, a type for sizes that can't be negative.

Compare it with an `int`, and `-Wall` warns you (below). Use `size_t` for the index instead (line 12).

```
6   int main() {
7       vector<string> artists = {"Charli XCX",
8                                 "100 gecs",
9                                 "A. G. Cook",
10                                "Danny L Harle"};
11
12      for (size_t i = 0; i < artists.size(); i++) {
13          cout << i << ": " << artists[i] << endl;
14      }
15
16      return 0;
17  }
```

the same loop with int i

```
$ g++ -Wall -Wextra indexLoop.cpp
indexLoop.cpp:12:23: warning: comparison of integer expressions of different signedness:
'int' and 'std::vector<std::__cxx11::basic_string<char> >::size_type' {aka 'long unsigned int'}
[-Wsign-compare]
```

Growing and shrinking: push_back, pop_back

`push_back(x)` adds `x` to the end. The vector grows by one.

`pop_back()` removes the last element.

No size to plan ahead, no `new`, no `delete`.

```
$ ./pushPop
size: 3
size: 5
Finn
Jake
Marceline
size: 3
```

```
6   int main() {
7       vector<string> heroes;  // starts empty
8
9       heroes.push_back("Finn");
10      heroes.push_back("Jake");
11      heroes.push_back("Marceline");
12      cout << "size: " << heroes.size() << endl;
13
14      heroes.push_back("BMO");
15      heroes.push_back("Gunter");
16      cout << "size: " << heroes.size() << endl;
17
18      heroes.pop_back();  // removes "Gunter"
19      heroes.pop_back();  // removes "BMO"
20
21      for (string hero : heroes) {
22          cout << hero << endl;
23      }
24      cout << "size: " << heroes.size() << endl;
25
26      return 0;
```

Predict: `[]` vs. `.at()`

`.at(i)` does the same job as `[i]`, with one difference.

Both lines 8 and 11 read index 4 of a 4-element vector. What happens at each?

```
5   int main() {
6       vector<int> victor = {0, 2, 4, 6};
7
8       cout << victor[4] << endl;
9       cout << "[] kept going" << endl;
10
11      cout << victor.at(4) << endl;
12      cout << ".at() kept going" << endl;
13
14      return 0;
15  }
```

Answer: `.at()` checks, `[]` doesn't

`[]` read whatever was in memory (here it happened to be 0) and kept going. `.at()` stopped the program with an **exception**: a message saying exactly what went wrong.

```
$ ./victor
0
[] kept going
terminate called after throwing an instance of 'std::out_of_range'
  what():  vector::_M_range_check: __n (which is 4) >= this->size() (which is 4)
Aborted
```

We like it when we get an actual run-time error instead of silent use of potential garbage. Exceptions get their own lesson later. For now, “the program stops and tells you why” is enough.

Other systems word the message differently, but they all name `out_of_range`.

Useful vector functions

Function	What it does
`size()`	How many elements are in the vector right now
`push_back(x)`	Adds `x` as a new element at the end
`pop_back()`	Removes the last element
`at(i)`	The element at index `i`, with bounds checking
`empty()`	`true` if the vector has no elements
`clear()`	Removes every element (size becomes 0)
`resize(n, x)`	Makes the size exactly `n`; any new elements get the value `x`

Full list: cppreference.com/w/cpp/container/vector

A vector keeps its elements on the heap

10 million ints in an array

```
4   int main() {
5       int scores[10000000];
6       scores[9999999] = 3;
7       cout << scores[9999999] << endl;
8       return 0;
9   }
```

```
$ ./bigArray
Segmentation fault
```

10 million ints in a vector

```
5   int main() {
6       vector<int> scores(10000000);
7       scores[9999999] = 3;
8       cout << scores[9999999] << endl;
9       return 0;
10  }
```

```
$ ./bigVector
3
```

The array is a local variable, so it lives on the small stack, and 40 MB doesn't fit. The vector puts its elements on the heap and does the `new` and `delete` for you.

Why plain arrays still matter

Every vector has one inside

A vector is a plain array on the heap, plus bookkeeping: its size and how much room it has left.

Text is arrays

`"hello"` is an array of 6 `char`s: the 5 letters plus a hidden end marker.

C code speaks arrays

`int main(int argc, char *argv[])` receives the command-line words as an array.

Iterators come from arrays

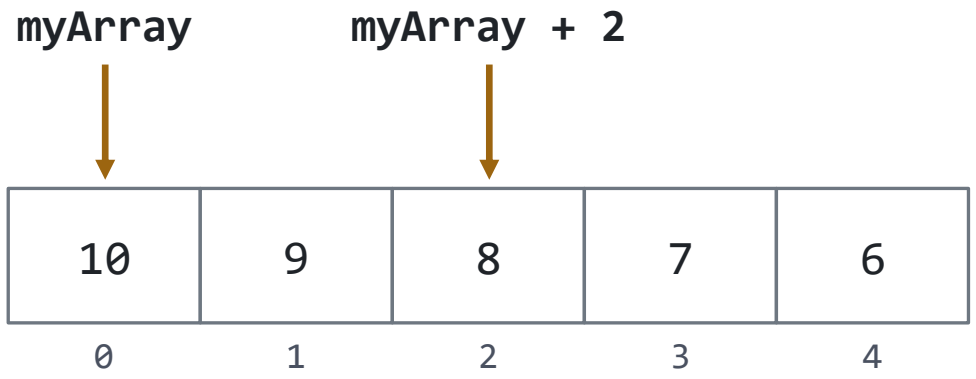
Stepping through an array with a pointer is where iterator syntax comes from. Coming up.

Use a `vector` by default. Know arrays, because everything else is built on them.

An array's name turns into a pointer

Almost anywhere you use it, an array's name turns into a pointer to its **first element**.

So `myArray[2]` means `*(myArray + 2)`: start at element 0, step forward 2, follow the pointer.




```
4   int main() {
5       int myArray[5];
6       for (int i = 0; i < 5; i++) {
7           myArray[i] = 10 - i;
8       }
9
10      cout << myArray[2] << endl;
11      cout << *(myArray + 2) << endl;
12
13      return 0;
14  }
```

```
$ ./arrayName
8
8
```

…and that's how an array loses its size

Inside `showSize()`, `numbers` is just a pointer, so `sizeof` gives the size of a pointer, not the array. g++ even warns about it.

That's why C functions always take a separate length. A `vector` never forgets its `size()`.

```
4    void showSize(int numbers[]) {
5        cout << "inside: " << sizeof(numbers) << endl;
6    }
7
8    int main() {
9        int scores[5] = {90, 85, 77, 92, 68};
10       cout << "in main: " << sizeof(scores) << endl;
11       showSize(scores);
12       return 0;
13   }
```

```
$ g++ -Wall arraySize.cpp && ./a.out
arraySize.cpp:5:34: warning: 'sizeof' on array function parameter
'numbers' will return size of 'int*' [-Wsizeof-array-argument]
in main: 20
inside: 8
```

Iterators

positions inside a container

Is it already in the vector?

Coming soon: you'll build a **Turing machine**, and it'll keep a vector of marked squares.

Before marking a square, you'll want to check: is it already in the vector?

So you look up the function for finding a value in a vector…



…and it takes things called **iterators**.

`find()`: first look

`find()` lives in `<algorithm>` (line 4). Forget that include and some compilers refuse to build this.

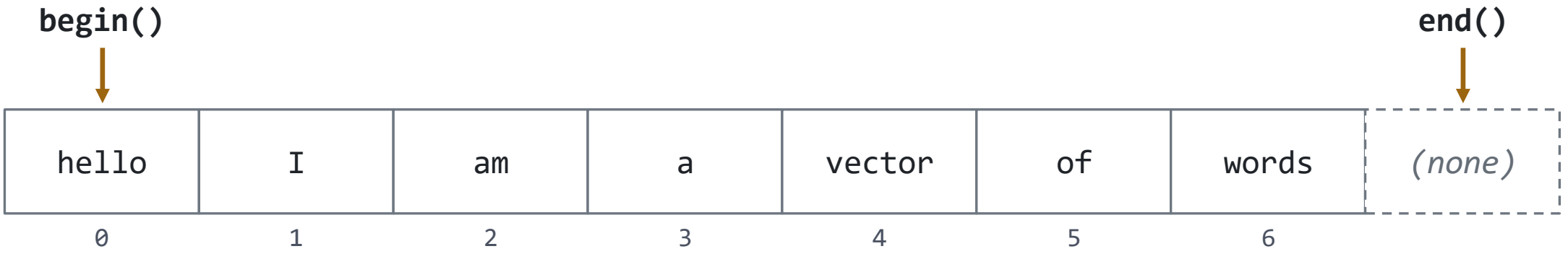
Line 12 searches `words` for `"vector"`. Line 14 checks whether it was found.

```
$ ./findWord
found at position 4
```

```
1   #include <iostream>
2   #include <string>
3   #include <vector>
4   #include <algorithm>
5   using namespace std;
6
7   int main() {
8       vector<string> words = {"hello", "I", "am", "a",
9                               "vector", "of", "words"};
10
11      vector<string>::iterator position;
12      position = find(words.begin(), words.end(), "vector");
13
14      if (position != words.end()) {
15          cout << "found at position "
16               << position - words.begin() << endl;
17      } else {
18          cout << "not found" << endl;
19      }
20
21      return 0;
22  }
```

What is an iterator?

An **iterator** marks a position in a container. It acts like a pointer: `*` gets the element there, `++` moves to the next one, and `==` / `!=` compare positions.



`begin()` is the first element. `end()` is **one past the last** element: there's nothing there, so never use `*` on it.

Every pointer into an array is an iterator. Not every iterator is a pointer.

Declaring an iterator, or letting `auto` do it

The full type

```
11      vector<string>::iterator position;
12      position = find(words.begin(), words.end(), "vector");
```

With auto

```
11      auto position = find(words.begin(), words.end(), "vector");
```

`vector<string>::iterator` means “an iterator into a `vector<string>`.” It's a mouthful.

`auto` tells the compiler to work out the type from the right-hand side. Same type, less typing.

How `find()` reports its answer

Found: an iterator to the **first** match.

Not found: `end()`. That's why line 14 compares with `words.end()`.

`position - words.begin()` turns a position into an index: “how many steps from the start?”

```
11      vector<string>::iterator position;
12      position = find(words.begin(), words.end(), "vector");
13
14      if (position != words.end()) {
15          cout << "found at position "
16               << position - words.begin() << endl;
17      } else {
18          cout << "not found" << endl;
19      }
```

```
$ ./findWord
found at position 4
```

```
$ ./findMissing
not found
```

Stepping off the end

`position + 1` is the next position. But if `position` is the **last** word, `position + 1` is `end()`, and `*end()` is undefined behavior.

Check before you step:

```
if (position + 1 != words.end())
```

searching for "words"

```
$ ./nextWord
found at position 6
Segmentation fault
```

```
11      auto position = find(words.begin(), words.end(),
12                           "words");
13
14      if (position != words.end()) {
15          cout << "found at position "
16               << position - words.begin() << endl;
17          cout << "next word: " << *(position + 1) << endl;
18      }
```

with the check

```
$ ./nextWordSafe
found at position 6
that was the last word
```

Iterators act like pointers

Same loop shape both times: **start**, **stop before the end**, **step**, and `*` to get the element.

Line 7 walks an array with a pointer. Line 13 walks a vector with an iterator.

A range-`for` does exactly this behind the scenes.

```
$ ./sameLoop
90 85 77
90 85 77
```

```
5   int main() {
6       int scores[] = {90, 85, 77};
7       for (int *p = scores; p != scores + 3; p++) {
8           cout << *p << " ";
9       }
10      cout << endl;
11
12      vector<int> nums = {90, 85, 77};
13      for (auto it = nums.begin(); it != nums.end(); it++) {
14          cout << *it << " ";
15      }
16      cout << endl;
17
18      return 0;
19  }
```

sort()

`sort()` also lives in `<algorithm>`. Give it a start and an end, and it puts everything in between in ascending order.

Arrays work too (line 15): pointers are iterators.

```
$ ./sortDemo
68 77 85 90 92
1 2 3
```

```
6   int main() {
7       vector<int> grades = {90, 85, 77, 92, 68};
8       sort(grades.begin(), grades.end());
9       for (int g : grades) {
10          cout << g << " ";
11      }
12      cout << endl;
13
14      int scores[] = {3, 1, 2};
15      sort(scores, scores + 3);  // pointers work too
16      for (int s : scores) {
17          cout << s << " ";
18      }
19      cout << endl;
20
21      return 0;
22  }
```

Predict: sorting words

What order do these four words end up in?

```
7   int main() {
8       vector<string> words = {"banana", "Zebra",
9                               "apple", "Mango"};
10      sort(words.begin(), words.end());
11      for (string w : words) {
12          cout << w << " ";
13      }
14      cout << endl;
15      return 0;
16  }
```

Answer: uppercase comes first

```
$ ./sortWords
Mango Zebra apple banana
```

Strings compare letter by letter, by **character code**, and every uppercase letter comes before every lowercase letter: `'Z'` is 90, `'a'` is 97.

Later you'll see how to give `sort()` your own rule, like “ignore case.”

`unique_ptr`

one heap object, one owner

Remember the ownership rule?

Every `new` has exactly one owner, who `delete`s it exactly once.

Forget, and you leak. Here, `main()` never deletes `meeting`:

```
140 int main() {
141     Clock *meeting = new Clock(9, 30, 0);
142     meeting->print24();
143
144     return 0;  // forgot to delete meeting
145 }
```

instructor demo (output trimmed, paths shortened)

```
$ g++ -fsanitize=address -g leakClock.cpp
$ ./a.out
09:30:00
==388==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 12 byte(s) in 1 object(s) allocated from:
    #1 0x5605d32a4b26 in main leakClock.cpp:141
SUMMARY: AddressSanitizer: 12 byte(s) leaked in 1 allocation(s).
```

`unique_ptr` owns its object

`make_unique<Clock>(9, 30, 0)` does the `new`. `->` works exactly like a regular pointer.

When `meeting` goes out of scope, the `unique_ptr` deletes the `Clock` for you. No `delete` anywhere, and no leak.

To use it: `#include <memory>`

```
140 int main() {
141     unique_ptr<Clock> meeting =
142         make_unique<Clock>(9, 30, 0);
143     meeting->print24();
144
145     return 0;  // no delete needed
146 }
```

instructor demo: the sanitizer finds nothing

```
$ g++ -fsanitize=address -g uniqueClock.cpp
$ ./a.out
09:30:00
```

You can't copy a `unique_ptr`

Copying would make two owners, so the compiler refuses. That's the ownership rule, enforced for you.

(Handing ownership to someone else *is* possible, with `std::move`. That's for later.)

```
140 int main() {
141     unique_ptr<Clock> meeting =
142         make_unique<Clock>(9, 30, 0);
143     unique_ptr<Clock> backup = meeting;
144
145     return 0;
146 }
```

```
$ g++ copyClock.cpp
copyClock.cpp:143:32: error: use of deleted function
'std::unique_ptr<_Tp, _Dp>::unique_ptr(const std::unique_ptr<_Tp, _Dp>&)
[with _Tp = Clock; _Dp = std::default_delete<Clock>]'
```

Which tool when?

If you need…	Use…
many values, and the count can change	a `vector`
a position inside a container	an iterator
one object on the heap	a `unique_ptr`
to see what's happening underneath	`new` and `delete` (what we do in this class)

Real-world C++ code rarely says `new` or `delete` directly. It uses tools like these.

Today's vocabulary

`vector`

An array that grows, knows its `size()`, and manages its own memory.

range-for

`for (string name : names)`: visit each element, no index.

`.at(i)`

Like `[i]`, but stops the program if `i` is out of range.

`size_t`

The type for sizes and indexes. Never negative.

iterator

A position in a container. Acts like a pointer.

`begin()` / `end()`

The first position, and one past the last.

`find()`

Returns the position of the first match, or `end()`.

`unique_ptr`

Owns one heap object and deletes it for you.